

Software Requirements Specification

Project Name: RangeFinder

Authors: Jan Francák, Jaroslav Vaněček, Jakub Šoma, Tomáš Elexa, Lola Rodríguez

Version: 1.3

Date: 2026-03-21

Contents

1. Introduction	4
1.1. Purpose	4
1.2. Document conventions	4
1.3. Intended Audience and Reading Suggestions	4
1.4. Product Scope	4
1.5. References	4
2. Overall Description	4
2.1. Product Perspective	4
2.2. Product Features/Functions	5
2.3. User Classes and Characteristics	5
2.4. Operating Environment	5
2.5. Design and Implementation Constraints	5
2.6. User Documentation	5
2.7. Assumptions and Dependencies	5
3. System scenarios (Use cases)	6
3.1. Use-case diagrams	6
3.2. Scenarios	6
3.2.1. SCN-1: User measures distance to an object	6
3.2.2. SCN-2: User displays measured data	7
3.2.3. SCN-3: User pairs Rangefinder through Bluetooth interface	7
3.2.4. SCN-4: User disconnects from Sensor Station	8
3.2.5. SCN-5: User configures units of measurement	9
3.2.6. SCN-6: User configures distance threshold	9
4. Functional Requirements	10
4.1. REQ-1 – Connecting to the SeS	10
4.2. REQ-2 – Displaying data	10
4.3. REQ-3 – Displaying the chart	10
4.4. REQ-4 – Displaying the Debug window	10
4.5. REQ-5 – Sending data	10
4.6. REQ-6 – Receiving data	11
4.7. REQ-7 – Detecting and measuring distance from objects	11
4.8. REQ-8 – Disconnecting from SeS	11
4.9. REQ-9 – Configuring units of measurement	11
4.10. REQ-10 – Configuring distance threshold	11
5. External Interface Requirements	11
5.1. User Interfaces (UI design / User Interaction Flow)	11
5.2. Hardware Interfaces	12
5.3. Software Interfaces (API Interfaces)	12
5.4. Communications Interfaces	12
6. Other Nonfunctional Requirements	12
6.1. Performance Requirements	12

6.2.	Safety Requirements	12
6.3.	Security Requirements	12
6.4.	Software Quality Attributes	12
7.	System Design & Architecture	13
7.1.	High-Level Architecture	13
7.2.	Technology Stack	13
7.3.	Data Flow	13
8.	Other Requirements	14

Figures

Figure 1	Context diagram of the RangeFinder system	4
Figure 2	Use-case diagram of the RangeFinder system	6
Figure 3	Component diagram of the RangeFinder system.	13

1. Introduction

1.1. Purpose

This document specifies the requirements for the Rangefinder, a Raspberry Pi based range-finding tool with a mobile phone app user interface. Its users are able to see the real time measurements of distance together with a chart to easily see any outliers.

1.2. Document conventions

- Requirement IDs: FREQ-x for functional requirements; NFREQ-x for nonfunctional requirements; SCN-x for scenarios.
- Time is stated in seconds; availability in %; data sizes in MB/GB
- RF – shortcut for Rangefinder – mobile application
- RPi – shortcut for Raspberry Pi
- SeS - shortcut for SensorStation, RPi with connected sensors
- BT - shortcut for Bluetooth

1.3. Intended Audience and Reading Suggestions

- Product owner / client: read Sections 1–3 for scope, users, and scenarios; skim 4 and 6 for priorities.
- Developers / architects: read Sections 2, 4–8; start with 7 for architecture constraints
- Testers / QA: read Sections 3, 4, 6 for traceability and acceptance criteria.
- Operations / security: read Sections 2.4–2.7 and 6.2–6.4

1.4. Product Scope

The Rangefinder provides a real time measurement of distance from the sensors mounted on the Raspberry Pi, that are transmitted by Bluetooth to a mobile phone app. Once the user pairs his app to the gadget, he will be able to see measurements of each sensor, the chart showing the measurements in a time period and a debug window of the Bluetooth connection. This version (1.3) focuses on the foundation of a more complex rangefinder, allowing easy further development.

1.5. References

Raspberry Pi: <https://www.raspberrypi.com>

2. Overall Description

2.1. Product Perspective

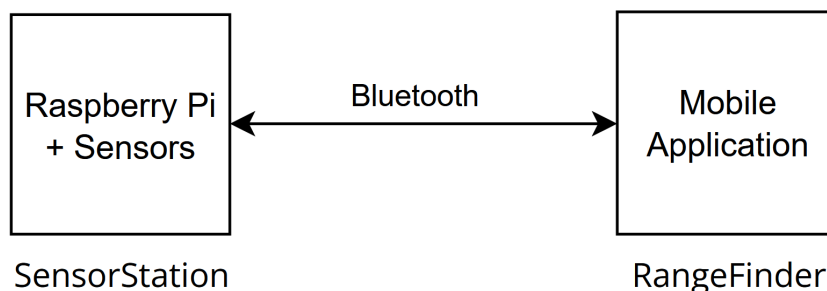


Figure 1: Context diagram of the RangeFinder system

Rangefinder is a new product based on the Raspberry PI helping with commercial-use distance measurement. Together with the Raspberry PI chassis is an in-house built phone application that shows all measured data. The context diagram shows a simplified look at the workings of the system.

2.2. Product Features/Functions

- Real-time measurements of each sensor
- Chart showing measurements over a set period of time
- Debug window for Bluetooth connection
- True/False obstacle checker
- Reliability evaluation of the sensor readings

2.3. User Classes and Characteristics

Not applicable

2.4. Operating Environment

- Raspberry Pi Zero 2W – Raspbian Lite
- Android operating system

2.5. Design and Implementation Constraints

- Inexperience with Bluetooth integration

2.6. User Documentation

- PDF User Manual
- PDF Troubleshooting Guide
- Release Notes per Version

2.7. Assumptions and Dependencies

- Stable Bluetooth connection

3. System scenarios (Use cases)

3.1. Use-case diagrams



Figure 2: Use-case diagram of the RangeFinder system

3.2. Scenarios

3.2.1. SCN-1: User measures distance to an object

FREQ reference	FREQ-7, FREQ-6, FREQ-5
NFREQ reference	NFREQ-3, NFREQ-1
Short Description:	User uses the SeS to measure a distance to an certain object
Actors:	User, SeS
Precondition:	SeS is turned on with Bluetooth enabled and SeS is paired with RF

Basic flow: User measures distance to an object

Step	User action	System response
1	Places an object in front of the ultrasonic / ToF sensor	The object is detected by the sensors
2	-	SeS measures the distance and sends it thorough Bluetooth interface
3	-	RF app recieves the data from SeS

Post-condition: SeS keeps sending new data as long as it is not turned off

Alternative flows

A1 – Object is out of bounds

- Trigger: Object is either too close or too far from the sensor
- System response: SeS sends either 0 or max value respectively
- Result: The measurement is inaccurate

A2 – Bluetooth connection is lost

- Trigger: The Bluetooth connection is unexpectedly lost
- System response: RF app alerts the user
- Result: User has to reconnect his BT connection

3.2.2. SCN-2: User displays measured data

FREQ reference	FREQ-2, FREQ-3, FREQ-4
NFREQ reference	NFREQ-3, NFREQ-4
Short Description:	User uses the Rangefinder app to read the data currently being sent by the sensors
Actors:	User
Precondition:	SeS connected and actively transmitting

Basic flow: Users displays measured data in the app

Step	User action	System response
1	SCN-1	SCN-1
2	-	RF app handles data processing and displays measurements in Simple or Advanced section of mobile app
3	Opens “Advanced” menu	“Advanced” menu gets opened
4	Views data	-

Post-condition: SeS keeps sending new data as long as it is not turned off

Alternative flows

A1 – Corrupted data

- Trigger: RF app receives a malformed payload
- System response: App catches the parsing exception and ignores the bad packet
- Result: App waits for the next packet without crashing

3.2.3. SCN-3: User pairs Rangefinder through Bluetooth interface

FREQ reference	FREQ-1
NFREQ reference	NFREQ-3
Short Description:	User uses the mobile app’s Bluetooth interface to pair the app to SeS
Actors:	User, SeS
Precondition:	SeS is on and advertising, phone has BT and location services enabled

Basic flow: User pairs Rangefinder through Bluetooth interface

Step	User action	System response
1	User selects the “Settings” section	Section opens
2	Turns scanning on by clicking on “Scan devices”	App starts scanning for pairable devices
3	-	App displays a list of available devices
4	User selects the SeS from the list	App attempts to connect to the selected device
5	-	Connection is successful, app starts receiving data from SeS

Post-condition: SeS and the app are successfully paired, and the app starts receiving data

Alternative flows

A1 – Malfunctioning SeS

- Trigger: SeS not working properly
- System response: SeS does not appear in the BT search
- Result: User cannot pair the app with SeS

A2 – Connection timeout

- Trigger: RF app fails to connect
- System response: App displays an error message
- Result: User is prompted to try again

3.2.4. SCN-4: User disconnects from Sensor Station

FREQ reference	FREQ-8
NFREQ reference	-
Short Description:	User terminates the connection to the SeS to save battery and stop the data stream
Actors:	User, SeS
Precondition:	RF app is currently connected to the SeS

Basic flow: User disconnects from Sensor Station

Step	User action	System response
1	User selects the “Settings” section	Section opens
2	User selects the “Disconnect” option	App explicitly unsubscribes from the BLE characteristic and closes the BT connection
3	-	App displays last known values in the data views, with a “Disconnected” tag
4	-	SeS detects the disconnection and goes back to advertising for new connections

Post-condition: SeS waits for another connection

Alternative flows

A1 – Hard exit

- Trigger: User forcefully kills the app
- System response: Mobile OS automatically severs the BLE connection
- Result: SeS detects the severing and goes back to broadcasting safely

3.2.5. SCN-5: User configures units of measurement

FREQ reference	FREQ-9, FREQ-2, FREQ-3
NFREQ reference	-
Short Description:	User changes the display units from meters to centimetres and inputs a new distance threshold
Actors:	User
Precondition:	RF app is currently connected to the SeS

Basic flow: User configures units of measurement

Step	User action	System response
1	User selects the “Settings” section	Section opens
2	User selects the “Units” option	App displays unit options
3	User selects the desired units for each sensor	App updates the display units

Post-condition: RF app is on operating on new units of measurement

3.2.6. SCN-6: User configures distance threshold

FREQ reference	FREQ-9, FREQ-2, FREQ-3
NFREQ reference	-
Short Description:	User sets a custom distance threshold for UI warnings.
Actors:	User
Precondition:	RF app is currently connected to the SeS

Basic flow: User configures distance threshold

Step	User action	System response
1	User selects the “Settings” section	Section opens
2	User selects the “Threshold” option	App displays threshold options
3	User inputs a new distance threshold for each sensor	App updates the threshold

Post-condition: RF app is sending UI warnings based on the new distance threshold

Alternative flows A1 – Invalid threshold input

- Trigger: User inputs a negative number, text or a value out of sensors physical bounds.
- System response: App informs the user of the invalid value and reverts to last known valid setting
- Result: The threshold is ready to be changed again

4. Functional Requirements

4.1. FREQ-1 – Connecting to the SeS

Description: The phone app should allow the user to connect to the SeS using Bluetooth

Priority: High

Acceptance criteria: Given the Bluetooth is working correctly on both devices, the user is able to connect to the SeS and starts to receive data.

4.2. FREQ-2 – Displaying data

Description: The phone app should allow the user to see data from each of the sensors, including the True/False obstacle detector.

Priority: High

Acceptance criteria: Given the user is connected to the SeS, they can clearly see data from each sensor in the “Advanced” menu of the app and min/max value in the “Simple” menu of the app.

4.3. FREQ-3 – Displaying the chart

Description: The phone app should allow the user to see data from all the sensors on the chart.

Priority: Medium

Acceptance criteria: Given the user is connected to the SeS, they can see the chart in the “Advanced” menu of the app and they can distinguish each sensor by their assigned colour.

4.4. FREQ-4 – Displaying the Debug window

Description: The phone app should allow the user to see the Debug window.

Priority: Medium

Acceptance criteria: Given the user is connected to the SeS, they can see the debug window on the bottom of the “Advanced” menu, where the entire Bluetooth connection between the devices is continuously displayed.

4.5. FREQ-5 – Sending data

Description: The SeS should begin data transmission once the BT connection is properly established.

Priority: High

Acceptance criteria: The data is being properly transmitted through the Bluetooth connection.

4.6. FREQ-6 – Receiving data

Description: The RF app should correctly receive the data, that are being sent through the Bluetooth connection.

Priority: High

Acceptance criteria: The data from the SeS are received and usable.

4.7. FREQ-7 – Detecting and measuring distance from objects

Description: The sensors should correctly measure distance from objects and send their data throughout the SeS to be transmitted through Bluetooth.

Priority: High

Acceptance criteria: The measured data from the sensors are being passed to the SeS.

4.8. FREQ-8 – Disconnecting from SeS

Description: The app should allow the user to disconnect from the SeS

Priority: Medium

Acceptance criteria: The app is successfully disconnected from the SeS

4.9. FREQ-9 – Configuring units of measurement

Description: The app should allow the user to configure measurement units between mm, cm, dm and m.

Priority: Low **Acceptance criteria:** The data are correctly recalculated and displayed.

4.10. FREQ-10 – Configuring distance threshold

Description: The app shall allow the user to configure distance threshold in currently chosen units of measurement.

Priority: Low

Acceptance criteria: If the object is closer or equals the threshold, a warning is displayed in the app.

5. External Interface Requirements

5.1. User Interfaces (UI design / User Interaction Flow)

- UI should include: Advanced menu, Simple menu, Settings menu, and appropriate navigation between them.
- Advanced menu should include:
 - A chart displaying data of each sensor in a set time period
 - Current measurement of each sensor in a separate box
 - Debug window of Bluetooth connection
 - Scrollbar to reach the debug window
- Simple menu should include:
 - Minimum distance currently measured by a sensor
 - Maximum distance currently measured by a sensor
 - Current data from the obstacle sensor

5.2. Hardware Interfaces

- I2C, GPIO pin readings
- Raspberry Zero 2W
- WaveShare Laser sensor
- Ultrasound sensor HC-SR04
- Time-of-flight sensor VL53LDK

5.3. Software Interfaces (API Interfaces)

- Go Bluetooth – BLE
- Bluetooth LE plugin for Xamarin & MAUI

5.4. Communications Interfaces

- Bluetooth with AES encryption

6. Other Nonfunctional Requirements

6.1. Performance Requirements

Not applicable

6.2. Safety Requirements

Not applicable

6.3. Security Requirements

The data transfer between SeS and RF app should be encrypted. For this purpose AES encryption will be used. The data on SeS will be encrypted by Secret Key, transmitted safely and then decrypted by the RF app.

6.4. Software Quality Attributes

1. **NFREQ-1 (Correctness):** The system shall measure within a meter accuracy.
2. **NFREQ-2 (Maintainability):** Codebase shall include automated tests covering at least 60% of backend logic and 40% of frontend components.
3. **NFREQ-3 (Portability):** The application shall work with any modern android phone.
4. **NFREQ-4 (Usability):** The application shall display all measured distances within at maximum half a second.

7. System Design & Architecture

7.1. High-Level Architecture

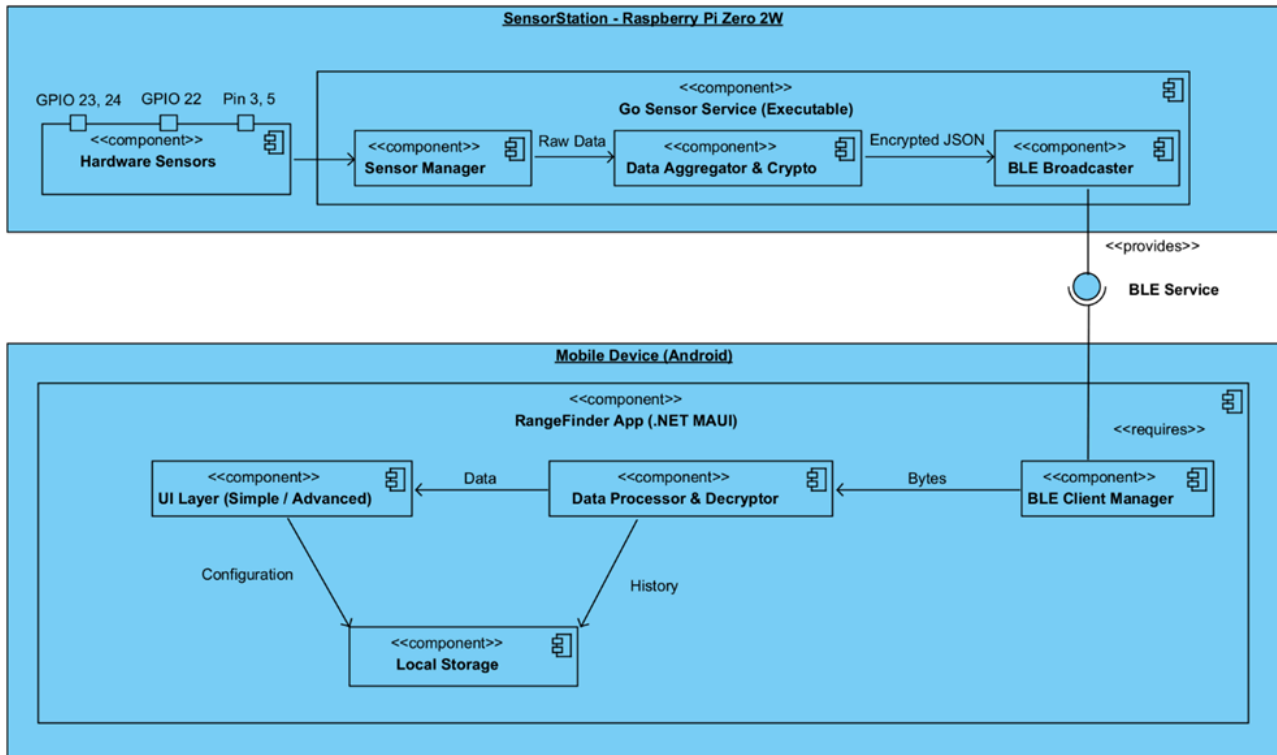


Figure 3: Component diagram of the RangeFinder system.

7.2. Technology Stack

- Phone App: .NET MAUI
- Hardware:
 - Main unit: Raspberry Pi Zero 2W
 - ARM64 architecture, 512 MB RAM, integrated BT chip
 - Operating system: Linux (Raspberry Pi OS, 64-bit)
 - Programming language: Go (Golang)
 - Architecture of runtime: Goroutines + Mutex – safe and effective asynchronous parallel processing
 - Key libraries:
 - go-rpio/v4 : low-level GPIO control
 - go-i2c & go-v15310x : communication with ToF sensor through HW registry
 - go-logger : for log control (I2C console spamming)
 - Communication and safety:
 - Bluetooth Low Energy (BLE) – GATT server using system BlueZ
 - E2E encryption: AES-256 in GCM mode
 - PSK Pre-Shared-Key key management through environment variables
 - Data structure: JSON

7.3. Data Flow

Data is collected from a single sensor → Data is transferred by Bluetooth to the mobile device application → Data is parsed and displayed in the application.

8. Other Requirements

None.